

Technische
Universität
Dresden

Project Report:

HW/SW Co-Design Lab

SS 2026

Faculty of Electrical Engineering and Computer Engineering
Institute of Communication Technology
Vodafone Chair of Mobile Communications Systems

Author: Muhammad Rehan Bukhari
Matriculation No.: 5315330
muhammad_rehan.bukhari@mailbox.tu-dresden.de
Supervisor: Shaown Mojumder
Date: 11.09.2026

Introduction

The HW/SW Co-Design Lab is intended to give a first practical access to this topic and experiencing the potential of the conjoint design of hardware and software. For this purpose, the reconfigurable processor flow from the company *Tensilica* is used (shown in fig. 1). The tool suite provides convenient means for analyzing and optimizing the performance of the software. Especially the possibility to easily extend the processor's hardware architecture by customer specific instructions using the *Tensilica Instruction Extension (TIE)* language is well suited to demonstrate the interaction between hardware and software design.

Within the scope of this lab, the following 2 exercises should be done:

1. Implementation of a finite impulse response (FIR) filter design including performance analysis; HW/SW optimization: e.g., by Fusion, SIMD extension, etc.; also consider different implementation alternatives for the FIR.
2. Enhance the performance of a given Vision Transformer (ViT) Feed-Forward Network (MLP) kernel using HW/SW co-design techniques such as Fusion, SIMD, FLIX, custom instructions, and other instruction extension methods.

The written report should only include the results of the second task and need not exceed 5 pages (source code excerpts not included)!

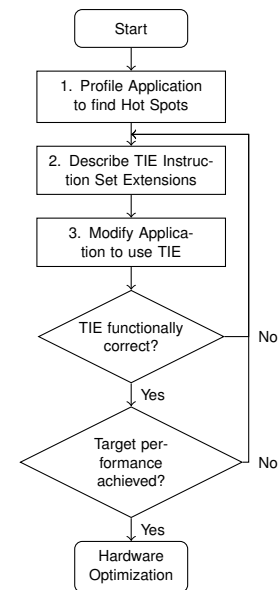


Figure 1: Similar to Tensilica Instruction Extension (TIE) Language, User's Guide. 02/2014, p. 4.

Source Code

Source code C files (only excerpts from the parts that have been changed; please highlight important parts).

The baseline `mlp_C` computes each of the three steps with a plain scalar loop; the three points marked `// Hint: Optimize here` are exactly the code that was replaced in later stages.

```
1  /* Step 1: FC1 */
2  for (i = 0; i < HIDDEN; i++) {
3      acc = b1[i];
4      // Hint: Optimize here
5      for (j = 0; j < IN_SIZE; j++) {
6          acc += (int32_t)in[j] * (int32_t)W1[i][j];
7      }
8      fc1[i] = acc;
9  }
10 /* Step 2: ReLU */
11 for (i = 0; i < HIDDEN; i++) {
12     // Hint: Optimize here
13     hidden[i] = relu(fc1[i]);
14 }
15 /* Step 3: FC2 */
16 for (i = 0; i < OUT_SIZE; i++) {
17     acc = b2[i];
18     // Hint: Optimize here
19     for (j = 0; j < HIDDEN; j++) {
20         acc += (int32_t)hidden[j] * (int32_t)W2[i][j];
21     }
22     out[i] = acc;
23 }
```

The final `mlp_C` kernel (the cache-based FLIX version) replaces all three scalar loops above with calls to custom TIE instructions. Highlighted parts (bold) mark the two optimizations added on top of the earlier SIMD/FLIX stages: filling all cache registers once per layer instead of once per neuron, and setting the weight pointer only once per layer since `W1/W2` are row-major and contiguous, so it auto-increments to the next row after streaming a full row. The scalar `relu()` from the baseline is likewise replaced by the `RELU()` TIE intrinsic, fused directly onto the MAC result.

```
1 void mlp_C(int8_t *in, int32_t *out)
2 {
3     int i;
4     int8_t hidden[HIDDEN];
5
6     /* Step 1: FC1 (input -> hidden pre-activation -> ReLU) */
7     WUR_ptr_in((unsigned)in);
8     MLP_FILL_CO();
```

```

9      MLP_FILL_C1();
10     MLP_FILL_C2();
11     MLP_FILL_C3();          /* entire input loaded into cache ONCE
    */
12
13     WUR_ptr_w((unsigned)&W1[0][0]); /* set weight pointer ONCE for the
    whole layer */
14
15     for (i = 0; i < HIDDEN; i++) {
16         WUR_mlp_acc(b1[i]);      /* bias still must be reloaded every
    neuron */
17         MLP_SEL_C0(); MLP_LD_W();
18         MAC_SIMD();   MLP_SEL_C1(); MLP_LD_W();
19         MAC_SIMD();   MLP_SEL_C2(); MLP_LD_W();
20         MAC_SIMD();   MLP_SEL_C3(); MLP_LD_W();
21         MAC_SIMD();
22
23         hidden[i] = RELU(RUR_mlp_acc());
24     }
25
26     /* Step 2: FC2 (hidden activation -> output) */
27     WUR_ptr_in((unsigned)hidden);
28     MLP_FILL_C0(); MLP_FILL_C1(); MLP_FILL_C2(); MLP_FILL_C3();
29     MLP_FILL_C4(); MLP_FILL_C5(); MLP_FILL_C6(); MLP_FILL_C7();
30
31     WUR_ptr_w((unsigned)&W2[0][0]); /* set weight pointer ONCE for the
    whole layer */
32
33     for (i = 0; i < OUT_SIZE; i++) {
34         WUR_mlp_acc(b2[i]);
35
36         MLP_SEL_C0(); MLP_LD_W();
37         MAC_SIMD();   MLP_SEL_C1(); MLP_LD_W();
38         MAC_SIMD();   MLP_SEL_C2(); MLP_LD_W();
39         MAC_SIMD();   MLP_SEL_C3(); MLP_LD_W();
40         MAC_SIMD();   MLP_SEL_C4(); MLP_LD_W();
41         MAC_SIMD();   MLP_SEL_C5(); MLP_LD_W();
42         MAC_SIMD();   MLP_SEL_C6(); MLP_LD_W();
43         MAC_SIMD();   MLP_SEL_C7(); MLP_LD_W();
44         MAC_SIMD();
45
46         out[i] = RUR_mlp_acc(); /* raw accumulator, no ReLU on output
    layer */
47     }
48 }

```

Source code TIE files

The listing below is the relevant excerpt of `Flix.tie`. Highlighted parts (bold) are the sliding-window cache states and the fill/select operation pair that removes the per-neuron memory reload (only C0/C1 are shown; C2-C7 follow the identical pattern with the corresponding cache register).

```

1  format mlp_flix 64 {slot_ld_in, slot_ld_w, slot_mac}
2
3  slot_opcodes slot_ld_in {MLP_LD_IN,

```

```

4           MLP_FILL_C0 , MLP_FILL_C1 , MLP_FILL_C2 ,
           MLP_FILL_C3 ,
5           MLP_FILL_C4 , MLP_FILL_C5 , MLP_FILL_C6 ,
           MLP_FILL_C7 ,
6           MLP_SEL_C0 ,  MLP_SEL_C1 ,  MLP_SEL_C2 ,
           MLP_SEL_C3 ,
7           MLP_SEL_C4 ,  MLP_SEL_C5 ,  MLP_SEL_C6 ,
           MLP_SEL_C7}
8 slot_opcodes slot_ld_w {MLP_LD_W}
9 slot_opcodes slot_mac {MAC_SIMD}
10
11 state ptr_in      32 add_read_write
12 state ptr_w      32 add_read_write
13 state mlp_acc     32 add_read_write
14
15 state mlp_in128_buf 128 add_read_write
16 state mlp_w128_buf  128 add_read_write
17
18 /* Sliding-window data cache: 8 x 128-bit registers.
19  * FC1 uses slots 0-3 to hold the entire 64-byte input vector.
20  * FC2 uses slots 0-7 to hold the entire 128-byte hidden vector.
21  * Filled ONCE per layer instead of once per neuron. */
22 state mlp_cache0 128 add_read_write
23 state mlp_cache1 128 add_read_write
24 /* mlp_cache2 .. mlp_cache7 declared identically */
25
26 operation MLP_LD_IN{}{out VAddr, in MemDataIn128,inout ptr_in, out
           mlp_in128_buf}{
27     assign VAddr = ptr_in;
28     assign ptr_in = ptr_in + 16;
29     assign mlp_in128_buf = MemDataIn128;
30 }
31
32 operation MLP_LD_W {}{out VAddr, in MemDataIn128,inout ptr_w,out
           mlp_w128_buf}{
33     assign VAddr = ptr_w;
34     assign ptr_w = ptr_w + 16;
35     assign mlp_w128_buf = MemDataIn128;
36 }
37
38 /* Cache fill: one-time load from memory into cache[k] */
39 operation MLP_FILL_C0{}{out VAddr, in MemDataIn128, inout ptr_in, out
           mlp_cache0}{
40     assign VAddr = ptr_in; assign ptr_in = ptr_in + 16; assign
           mlp_cache0 = MemDataIn128;
41 }
42 operation MLP_FILL_C1{}{out VAddr, in MemDataIn128, inout ptr_in, out
           mlp_cache1}{
43     assign VAddr = ptr_in; assign ptr_in = ptr_in + 16; assign
           mlp_cache1 = MemDataIn128;
44 }
45 /* MLP_FILL_C2 .. MLP_FILL_C7 identical, targeting mlp_cache2 ..
           mlp_cache7 */
46
47 /* Cache select: zero-latency register copy, no memory access */
48 operation MLP_SEL_C0{}{in mlp_cache0, out mlp_in128_buf}{ assign
           mlp_in128_buf = mlp_cache0; }
49 operation MLP_SEL_C1{}{in mlp_cache1, out mlp_in128_buf}{ assign

```

```

    mlp_in128_buf = mlp_cache1; }
50 /* MLP_SEL_C2 .. MLP_SEL_C7 identical, reading mlp_cache2 .. mlp_cache7
    */
51
52 operation MAC_SIMD{{inout mlp_acc, in mlp_in128_buf,in mlp_w128_buf}}{
53     wire [15:0] tp0 = TIEmul(mlp_in128_buf[7:0], mlp_w128_buf[7:0], 1'
        b1);
54     wire [15:0] tp1 = TIEmul(mlp_in128_buf[15:8], mlp_w128_buf[15:8], 1'
        b1);
55     /* tp2 .. tp15: identical 8-bit x 8-bit multiplies over the
56        * remaining 14 byte lanes of the 128-bit input/weight buffers */
57
58     wire [31:0] p0 = {{16{tp0[15]}},tp0};
59     wire [31:0] p1 = {{16{tp1[15]}},tp1};
60     /* p2 .. p15: sign-extend the remaining 14 partial products */
61
62     assign mlp_acc = TIEaddn(mlp_acc,p0,p1,p2,p3,p4,p5,p6,p7,p8,p9,p10,
        p11,p12,p13,p14,p15);
63 }
64
65 operation RELU {out AR res, in AR val}{{
66     wire [31:0] rel = val[31]? 32'b0: val;
67     wire [31:0] div = rel >> 7;
68     assign res = (|div[31:7])? 32'd127: div;
69 }

```

Report Questions

Baseline Implementation

Explain the baseline MLP implementation.

- Describe the computation flow of $\text{FC1} \rightarrow \text{ReLU} \rightarrow \text{FC2}$.
- Which parts of the code take the most execution time?
- Which loops are executed most frequently?

- **Computation flow ($\text{FC1} \rightarrow \text{ReLU} \rightarrow \text{FC2}$):** FC1 multiplies the input vector by W_1 and adds b_1 ; ReLU clips negative activations to zero; FC2 multiplies the ReLU output by W_2 and adds b_2 to produce the output. Each FC layer is a nested loop: outer loop over output neurons, inner loop performs a scalar multiply-accumulate (MAC) over input elements.
- **Parts taking the most execution time:** Almost all runtime is inside `mlp_C`: 552,087 of 559,326 total cycles (98.7%), of which 547,095 cycles (97.8% of the total) are the function's own body. The remaining 1.3% is fixed `_ResetHandler`/C-runtime startup overhead.
- **Loops executed most frequently:** The FC1 and FC2 inner MAC loops execute most frequently, and equally: each runs $\text{HIDDEN} \times \text{IN_SIZE} = 128 \times 64 = 8,192$ iterations, since W_1 (128×64) and W_2 (64×128) hold the same number of weights. These 16,384 MAC iterations dominate the baseline runtime, far outweighing the 128-iteration ReLU loop, and are the hot spot targeted by all later optimization stages.

Optimization Strategy and HW/SW Co-Design

Describe the optimization methods used in your implementation.

- What software optimizations were applied?
- What hardware acceleration methods or custom instructions were used?
- Why were these optimizations selected?
- How did the software and hardware optimizations work together?
- Which optimization provided the largest performance improvement?

- **Software:** Full unrolling of the small, constant-trip inner loops (4 chunks FC1, 8 FC2); software pipelining (chunk j 's MAC alongside chunk $j+1$'s loads); removal of a redundant per-neuron weight-pointer reset via the row-major contiguity of W_1/W_2 .
- **Hardware / custom instructions:** Existing `MAC_SIMD` (16-lane 8×8 multiply, 17-input adder tree, one cycle) and 3-slot FLIX bundle. Added: an 8-slot, 128-bit sliding-window cache (`mlp_cache0..7`) with `MLP_FILL_Ck` (load once per layer) and `MLP_SEL_Ck` (zero-latency copy).
- **Why selected:** Each targets a distinct waste source: pipelining fills idle FLIX slots; the cache removes redundant transactions (vector re-fetched per neuron instead of per layer); the pointer fix removes dead instructions found via the weight layout. None change the underlying arithmetic.

- **HW/SW co-design:** Hardware supplies the primitives; software decides when to invoke them. The cache saves nothing without `mlp.c`'s "fill once, select many" pattern; no software change alone cuts the $16\times$ MAC-instruction count without `MAC_SIMD` in hardware.
- **Largest improvement:** By transactions, the cache is largest, removing $\approx 49\%$ (1012 of 2048). By cycles, pipelining plus cache (-02) took `mlp_C` from 3,786 to 2,846; the same code at -00 gave 9,949, worse than baseline – the gain needs compiler scheduling.

Computational Efficiency

Explain how your implementation improves computational efficiency.

- Were there opportunities to reduce unnecessary computations?
- Did your implementation reduce unnecessary memory accesses or operations?
- Explain how your optimization improved execution efficiency.
- Hint: Carefully analyze the workload and dataset structure.

- **Reducing computations:** Not in FLOP count – this is a dense MLP, every weight genuinely contributes, so no MAC was unnecessary. Savings came from *how* MACs were issued: $16\times$ fewer instructions via SIMD, and removing pointer resets that recomputed a value auto-increment already held.
- **Reducing memory accesses:** Yes, substantially. The cache cut memory transactions from 2,048 to 1,036 ($\approx 49\%$) by recognizing the input/hidden vector is identical for every neuron in a layer, so it only needs fetching once per layer. The pointer fix removed a further 190 instructions recomputing an already-reached address.
- **Efficiency gained:** Bandwidth (far fewer bytes moved for the same result) and issue efficiency (more useful work per FLIX bundle once adjacent loads/compute no longer serialize).
- **Workload structure:** The key insight is a reuse asymmetry: the activation vector is broadcast (read identically by every neuron in a layer), while the weight matrix is streamed (each row read once) – a dense matrix-vector product $W \cdot x$ where x is reused and W is not. This motivated caching the activation vector, not the weights. Separately, `W1/W2` being row-major and contiguous enabled the pointer-reset elimination.

Performance Evaluation

Evaluate the performance of your implementation.

- Compare execution cycles before and after optimization.
- Report the achieved speedup.
- If performance decreased in any step, explain why.
- Which part of the implementation is still the main bottleneck?

Cycles and speedup:

Build	<code>mlp_C</code> cycles	Speedup
Baseline (scalar C, -00)	552,087	1.00x
Pipelined FLIX (no cache)	3,786	145.8x
Cache-based FLIX (final)	2,846	194.0x

The big jump comes from moving to FLIX in the first place. The cache on top of that is a smaller, further win: it takes `mlp_C` from 3,786 down to 2,846 cycles, about 25% off what was already a fast build.

- **Where performance decreased:** No regression was seen between these three builds; each stage was strictly faster than the one before it.

- **Remaining bottleneck:** With the kernel this fast, the bottleneck has shifted entirely, a textbook case of Amdahl's Law: the fixed `_ResetHandler/C-runtime` startup cost (3,906 cycles) barely registered in the 552,087-cycle baseline, but in the final 8,996-cycle total it now accounts for 43.4 % of all runtime, more than the optimized kernel itself. Within `mlp_C` specifically, what remains is the weight stream: every neuron's weight row is unique and read once, so there is nothing left to cache there. Secondly, the `RUR_mlp_acc() → RELU() → store tail` per FC1 neuron cannot overlap with that neuron's own MAC work (a fusion attempt was tried and reverted).

Area, Trade-offs, and Correctness

Evaluate the hardware cost and overall implementation trade-offs.

- What is the hardware area impact of your optimizations?
- Discuss the trade-off between performance and hardware cost.
- Explain how correctness was verified using the reference output.
- Which optimization provided the best speed/area trade-off?

- **Area impact:** On the 150,000-gate base core: the main-task TIE optimizations (MAC fusion, SIMD, dual DMEM, fused ReLU/quant) add 30,691 gates (20.5 %) for 9.21x. The bonus FLIX track adds 13,024 gates (about 8 %) for 145.8x in its pipelined-only form, rising to 29,997 gates (about 16.7 %) for 194.0x once the sliding-window cache is added.
- **Performance vs. cost:** Diminishing returns within the FLIX track: the cache-based version spends 130 % more gates than the pipelined-only version for only 33 % more speedup. Pipelining and the pointer fix cost zero extra gates, so they were applied first.
- **Correctness verification:** Checked on the ISS (`xt-run`): outputs compared against `ref_out[]`, printing "Results match." on success, confirmed for every stage including the final cache-based FLIX version.
- **Best speed/area trade-off:** The pointer-reset fix: 190 instructions removed for zero gates. Among area-costing changes, the pipelined-only FLIX version (13,024 gates, 145.8x) trades better than the cache-based version; the cache-based version was still kept as final since cycle count, not area efficiency, is the primary metric.